

Sesión 14 (D) — La matriz de regresores en la práctica: verificación con datos simulados

Marcos Bujosa

Descripción de la práctica

Cerramos la sesión de regresión múltiple volviendo, por primera vez con Gretl, sobre la notación matricial de la lección 10: la matriz de datos $\mathbf{X}$, las ecuaciones normales $\mathbf{X}^\top \mathbf{X} \hat{\beta} = \mathbf{X}^\top \mathbf{y}$, y la condición de ortogonalidad $\mathbf{X}^\top \hat{e} = \mathbf{0}$. Trabajamos con datos *simulados* porque ahora no queremos *interpretar* ningún fenómeno económico; queremos *verificar* que la maquinaria algebraica de la lección 10 funciona exactamente como se demostró: primero en un caso general, y después en el caso especial de regresores mutuamente ortogonales (“familia ortogonal”), donde cada coeficiente se puede calcular de forma aislada.

Objetivo

1. Construir explícitamente, con el comando `matrix` de Gretl, la matriz de datos $\mathbf{X}$ de un modelo con dos regresores no constantes.
2. Calcular $\mathbf{X}^\top \mathbf{X}$ y $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$, comparando el resultado con el que da `ols` directamente.
3. Verificar $\mathbf{X}^\top \hat{e} = \mathbf{0}$, en sus dos lecturas (matriz por vector y vector por matriz).
4. Construir un caso de *familia ortogonal* (regresores mutuamente ortogonales) y comprobar que cada $\hat{\beta}_j$ se puede calcular de forma aislada, sin resolver ningún sistema.
5. Verificar, en ese mismo caso, que $R^2 = \rho_{z_2 y}^2 + \rho_{z_3 y}^2$.

Actividad 1 - Datos simulados: un caso general, no ortogonal

Generamos $n = 40$ observaciones con dos regresores no constantes, x_2 y x_3 , deliberadamente *correlacionados entre sí* (para no caer, sin darnos cuenta, en el caso especial de la Actividad 4).

en línea de comandos:

```
nulldata 40 --preserve
set seed 97531

series x2 = uniform(0,10)
series x3 = 0.6*x2 + normal(0,2)
series u = normal(0,3)

scalar b1 = 5
scalar b2 = 2
scalar b3 = -1
series y = b1 + b2*x2 + b3*x3 + u

printf "Correlacion entre x2 y x3 = %.4f (no son ortogonales)\n", corr(x2,x3)
```

Recuerde la advertencia de siempre sobre los vectores de diseño: $\mathbf{u}$ es el vector que nosotros usamos para generar los datos; no debe confundirse con el vector de residuos $\hat{e}$ que calculará más adelante MCO a partir de los datos simulados.

Licencia: Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0).

Actividad 2 - La matriz de regresores y las ecuaciones normales

Construimos explícitamente la matriz de datos $\mathbf{X} = [1; \mathbf{x}_2; \mathbf{x}_3]$ (lección 10) con el comando `matrix` de Gretl, a partir de una lista de series.

en línea de comandos:

```
list Xlist = const x2 x3
matrix X = { Xlist }
list ylist = y
matrix Y = { ylist }

matrix XtX = X'X
matrix XtY = X'Y

printf "Matriz X'X (3x3):\n"
print XtX
printf "\nVector X'y (3x1):\n"
print XtY
```

Matriz X'X (3x3):
XtX (3 x 3)

40,000	193,05	114,98
193,05	1290,6	715,42
114,98	715,42	612,46

Vector X'y (3x1):
XtY (3 x 1)

472,54
2881,9
1421,0

Ahora resolvemos las ecuaciones normales *usando la inversa* —tal como aparece en la mayoría de los manuales (lección 10)— y comparamos el resultado con lo que da `ols` directamente.

en línea de comandos:

```
matrix beta_matricial = invpd(XtX)*XtY
printf "beta calculado como (X'X)^(-1) X'y:\n"
print beta_matricial

ols y const x2 x3 --quiet
matrix beta_ols = $coeff
printf "\nbeta calculado por 'ols' de Gretl:\n"
print beta_ols

matrix diferencia = beta_matricial - beta_ols
printf "\nDiferencia (debe ser practicamente cero):\n"
print diferencia
```

beta calculado como $(\mathbf{X}'\mathbf{X})^{-1} \mathbf{X}'\mathbf{y}$:
beta_matricial (3 x 1)

4,4307
2,1139
-0,98078

beta calculado por 'ols' de Gretl:
beta_ols (3 x 1)

4,4307
2,1139
-0,98078

Diferencia (debe ser practicamente cero):
diferencia (3 x 1)

```
2,6645e-15
4,4409e-16
-1,3323e-15
```

Recuerde la advertencia de la lección 10: esta fórmula con inversa EXISTE, y es exactamente lo que Gretl calcula “por dentro” al ejecutar `ols`; pero el camino conceptual del curso nunca ha sido, ni será, invertir una matriz: es siempre la condición de ortogonalidad $\mathbf{X}^\top \hat{\mathbf{e}} = \mathbf{0}$, que verificamos en la siguiente actividad.

Actividad 3 - Las dos lecturas de la ortogonalidad: $\mathbf{X}^\top \hat{\mathbf{e}}$ y $\hat{\mathbf{e}}\mathbf{X}$

Recuperamos el vector de residuos de la última regresión y verificamos que es ortogonal a las tres columnas de $\mathbf{X}$ simultáneamente, en sus dos lecturas equivalentes (lección 10, “Transposición”).

en línea de comandos:

```
series ehat = $uhat
list elist = ehat
matrix E = { elist }

matrix XtE = X'E
printf "X'e (columna 3x1, debe ser (aprox.) el vector nulo):\n"
print XtE

matrix eX = E'X
printf "\ne*X (fila 1x3, el mismo resultado, transpuesto):\n"
print eX
```

X'e (columna 3x1, debe ser (aprox.) el vector nulo):
XtE (3 x 1)

```
-1,5543e-13
5,7732e-13
-4,0945e-13
```

e*X (fila 1x3, el mismo resultado, transpuesto):
eX (1 x 3)

```
-1,5543e-13   5,7732e-13  -4,0945e-13
```

Observe que $\mathbf{XtE}$ y $\mathbf{eX}$ contienen *los mismos tres números*, solo que uno en forma de columna y el otro en forma de fila: es exactamente la identidad $\hat{\mathbf{e}}\mathbf{X} = \mathbf{X}^\top \hat{\mathbf{e}}$ de la lección 10, comprobada aquí numéricamente.

Actividad 4 - Familia ortogonal: un diseño con regresores mutuamente ortogonales

Construimos ahora un caso muy distinto: dos regresores, $\mathbf{z}_2$ y $\mathbf{z}_3$, diseñados a propósito para ser *mutuamente ortogonales* (y de media nula), siguiendo un patrón de diseño factorial con $n = 8$.

en línea de comandos:

```
nulldata 8 --preserve
setobs 1 1 --cross-section
set seed 24601

matrix mz2 = {-1;-1;-1;-1;1;1;1;1}
matrix mz3 = {-1;-1;1;1;-1;-1;1;1}
```

```

series z2 = mz2
series z3 = mz3

printf "suma(z2)      = %.4f      (debe ser 0: media nula)\n", sum(z2)
printf "suma(z3)      = %.4f      (debe ser 0: media nula)\n", sum(z3)
printf "suma(z2*z3)    = %.4f      (debe ser 0: ortogonalidad mutua)\n", sum(z2*z3)

scalar c1 = 10
scalar c2 = 3
scalar c3 = -2
series u2 = normal(0,1)
series y2 = c1 + c2*z2 + c3*z3 + u2

```

```

suma(z2)      = 0,0000      (debe ser 0: media nula)
suma(z3)      = 0,0000      (debe ser 0: media nula)
suma(z2*z3)    = 0,0000      (debe ser 0: ortogonalidad mutua)

```

Note que z_2 y z_3 valen siempre ± 1 : es un diseño “de juguete” (parecido a los que reaparecerán, con otro propósito, cuando estudiemos variables dummy), elegido únicamente para lograr una ortogonalidad EXACTA (no aproximada) entre ambos regresores.

Actividad 5 - Cada coeficiente por separado: la familia ortogonal en acción

Recordando la transparencia “Familia ortogonal” de la lección 10: si los regresores son mutuamente ortogonales, cada $\hat{\beta}_j$ puede calcularse *de forma aislada*, sin resolver ningún sistema, con la misma fórmula de Cauchy–Schwarz de la lección 3. Verificamos esto comparando dos vías de cálculo completamente independientes.

en línea de comandos:

```

ols y2 const z2 z3 --quiet
scalar b2_conjunto = $coeff(z2)
scalar b3_conjunto = $coeff(z3)
scalar r2_conjunto = $rsq

scalar b2_aislado = cov(z2,y2)/var(z2)
scalar b3_aislado = cov(z3,y2)/var(z3)

scalar rho2 = corr(z2,y2)
scalar rho3 = corr(z3,y2)
scalar suma_rho2 = rho2^2 + rho3^2

printf "beta2: conjunto (ols z2 z3) = %.6f      aislado (cov/var) = %.6f\n", b2_conjunto, b2_aislado
printf "beta3: conjunto (ols z2 z3) = %.6f      aislado (cov/var) = %.6f\n", b3_conjunto, b3_aislado
printf "\nR2 (modelo conjunto)          = %.6f\n", r2_conjunto
printf "rho(z2,y)^2 + rho(z3,y)^2        = %.6f\n", suma_rho2

```

```

beta2: conjunto (ols z2 z3) = 2,952111      aislado (cov/var) = 2,952111
beta3: conjunto (ols z2 z3) = -2,488857      aislado (cov/var) = -2,488857

R2 (modelo conjunto)          = 0,968065
rho(z2,y)^2 + rho(z3,y)^2      = 0,968065

```

Los dos cálculos de cada coeficiente coinciden (salvo errores de redondeo), y R^2 coincide con la suma de las dos correlaciones al cuadrado: exactamente lo que predice la lección 10 para este caso especial. Compare este resultado con el caso general de la Actividad 1, donde x_2 y x_3 estaban correlacionados entre sí: allí no habría tenido ningún sentido calcular $\hat{\beta}_2$ ignorando a x_3 , porque el resultado NO habría coincidido con el coeficiente del modelo conjunto.

Actividad 6 - Síntesis

Hoy hemos verificado con Gretl, en dos escenarios bien distintos, la maquinaria matricial completa de la lección 10: (i) en el caso general, que $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ coincide con lo que calcula `ols`, y que $\mathbf{X}^\top \hat{\mathbf{e}} = \mathbf{0}$ (que es lo mismo que $\hat{\mathbf{e}}\mathbf{X} = \mathbf{0}$); (ii) en el caso especial de regresores mutuamente ortogonales, que cada coeficiente puede aislarse sin resolver ningún sistema, y que R^2 se descompone como suma de correlaciones al cuadrado es ese caso tan especial.

Preguntas de interpretación para la clase

1. En la Actividad 2, ¿por qué la diferencia entre `beta_matricial` y `beta_ols` no es exactamente cero, sino un número muy pequeño?
2. ¿Por qué necesitamos que `x2` y `x3` estén correlacionados en la Actividad 1 para que el ejercicio tenga sentido pedagógico? ¿Qué habría pasado si, por descuido, hubieran resultado casi ortogonales?
3. En la Actividad 4, ¿por qué es importante que `z2` y `z3` tengan media *exactamente* cero, y no solo estén incorrelados?
4. Si en la Actividad 5 hubiéramos calculado $\hat{\beta}_2$ “aislado” (`cov(z2,y2)/var(z2)`) usando los datos de la Actividad 1 (`x2`, `x3`, correlacionados), ¿coincidiría con el coeficiente de `x2` en el modelo conjunto `ols` y `const x2 x3`? ¿Por qué sí o por qué no?
5. ¿Diría que el uso de la inversa $(\mathbf{X}^\top \mathbf{X})^{-1}$ en esta práctica contradice la advertencia de la lección 10 de que el curso “nunca la usa como vía de razonamiento”? ¿Qué diferencia hay entre usarla para *verificar* una fórmula ya conocida y usarla como *método* para resolver MCO?

Para profundizar:

- Véase la lección 10 ([S13-Lecc10](#)) para la notación matricial completa, el operador selector y la transparencia “Familia ortogonal”.
- Cottrell, A. y Lucchetti, R. (2023). *Gretl User's Guide*. Sección sobre construcción y operaciones con matrices a partir de listas de series.

Código completo de la práctica

```
# -----
# Copyright (C) 2026 Marcos Bujosa
# Licencia: GNU General Public License v3.0 o posterior
# Este código es software libre y puede ser redistribuido y/o modificado bajo los términos de la GPL.
# Ver el archivo LICENSE del repositorio para más detalles.
# -----

# Los dos primeros comandos son necesarios para que Gretl guarde los resultados de la práctica en el
↳ directorio de trabajo
# al ejecutar lo siguiente desde un terminal (use los nombres y ruta que correspondan)
#
#   DIRECTORIO="Nombre_Directorio_trabajo" gretlcli -b ruta/nombre_fichero_de_la_practica.inp
#
# Si esto no le funciona en su sistema, comente las siguientes dos líneas y sitúese en el directorio de
↳ trabajo de gretl
# que corresponda (configure dicho directorio de trabajo desde la ventana principal de Gretl).

string directory = getenv("DIRECTORIO")
set workdir "@directory"

nulldata 40 --preserve
set seed 97531

series x2 = uniform(0,10)
series x3 = 0.6*x2 + normal(0,2)
series u = normal(0,3)

scalar b1 = 5
scalar b2 = 2
scalar b3 = -1
series y = b1 + b2*x2 + b3*x3 + u

printf "Correlacion entre x2 y x3 = %.4f (no son ortogonales)\n", corr(x2,x3)

list Xlist = const x2 x3
matrix X = { Xlist }
list ylist = y
matrix Y = { ylist }

matrix XtX = X'X
matrix XtY = X'Y

printf "Matriz X'X (3x3):\n"
print XtX
printf "\nVector X'y (3x1):\n"
print XtY

outfile --quiet matrizXyEcuacionesNormales.txt
    list Xlist = const x2 x3
    matrix X = { Xlist }
    list ylist = y
    matrix Y = { ylist }

    matrix XtX = X'X
    matrix XtY = X'Y

    printf "Matriz X'X (3x3):\n"
    print XtX
    printf "\nVector X'y (3x1):\n"
    print XtY
end outfile

matrix beta_matricial = invpd(XtX)*XtY
printf "beta calculado como (X'X)^(-1) X'y:\n"
```

```

print beta_matricial

ols y const x2 x3 --quiet
matrix beta_ols = $coeff
printf "\nbeta calculado por 'ols' de Gretl:\n"
print beta_ols

matrix diferencia = beta_matricial - beta_ols
printf "\nDiferencia (debe ser practicamente cero):\n"
print diferencia

outfile --quiet betaMatricialVsOls.txt
    matrix beta_matricial = invpd(XtX)*XtY
    printf "beta calculado como (X'X)^(-1) X'y:\n"
    print beta_matricial

    ols y const x2 x3 --quiet
    matrix beta_ols = $coeff
    printf "\nbeta calculado por 'ols' de Gretl:\n"
    print beta_ols

    matrix diferencia = beta_matricial - beta_ols
    printf "\nDiferencia (debe ser practicamente cero):\n"
    print diferencia
end outfile

series ehat = $uhat
list elist = ehat
matrix E = { elist }

matrix XtE = X'E
printf "X'e (columna 3x1, debe ser (aprox.) el vector nulo):\n"
print XtE

matrix eX = E'X
printf "\ne*X (fila 1x3, el mismo resultado, transpuesto):\n"
print eX

outfile --quiet verificacionXteYeX.txt
    series ehat = $uhat
    list elist = ehat
    matrix E = { elist }

    matrix XtE = X'E
    printf "X'e (columna 3x1, debe ser (aprox.) el vector nulo):\n"
    print XtE

    matrix eX = E'X
    printf "\ne*X (fila 1x3, el mismo resultado, transpuesto):\n"
    print eX
end outfile

nulldata 8 --preserve
setobs 1 1 --cross-section
set seed 24601

matrix mz2 = {-1;-1;-1;-1;1;1;1;1}
matrix mz3 = {-1;-1;1;1;-1;-1;1;1}
series z2 = mz2
series z3 = mz3

printf "suma(z2)      = %.4f      (debe ser 0: media nula)\n", sum(z2)
printf "suma(z3)      = %.4f      (debe ser 0: media nula)\n", sum(z3)
printf "suma(z2*z3)    = %.4f      (debe ser 0: ortogonalidad mutua)\n", sum(z2*z3)

scalar c1 = 10
scalar c2 = 3

```

```

scalar c3 = -2
series u2 = normal(0,1)
series y2 = c1 + c2*z2 + c3*z3 + u2

outfile --quiet datosFamiliaOrtogonal.txt
    nulldata 8 --preserve
    setobs 1 1 --cross-section
    set seed 24601

    matrix mz2 = {-1;-1;-1;-1;1;1;1;1}
    matrix mz3 = {-1;-1;1;1;-1;-1;1;1}
    series z2 = mz2
    series z3 = mz3

    printf "suma(z2)      = %.4f      (debe ser 0: media nula)\n", sum(z2)
    printf "suma(z3)      = %.4f      (debe ser 0: media nula)\n", sum(z3)
    printf "suma(z2*z3) = %.4f      (debe ser 0: ortogonalidad mutua)\n", sum(z2*z3)

    scalar c1 = 10
    scalar c2 = 3
    scalar c3 = -2
    series u2 = normal(0,1)
    series y2 = c1 + c2*z2 + c3*z3 + u2
end outfile

outfile --quiet familiaOrtogonalComparacion.txt
    ols y2 const z2 z3 --quiet
    scalar b2_conjunto = $coeff(z2)
    scalar b3_conjunto = $coeff(z3)
    scalar r2_conjunto = $rsq

    scalar b2_aislado = cov(z2,y2)/var(z2)
    scalar b3_aislado = cov(z3,y2)/var(z3)

    scalar rho2 = corr(z2,y2)
    scalar rho3 = corr(z3,y2)
    scalar suma_rho2 = rho2^2 + rho3^2

    printf "beta2: conjunto (ols z2 z3) = %.6f      aislado (cov/var) = %.6f\n", b2_conjunto, b2_aislado
    printf "beta3: conjunto (ols z2 z3) = %.6f      aislado (cov/var) = %.6f\n", b3_conjunto, b3_aislado
    printf "\nR2 (modelo conjunto)          = %.6f\n", r2_conjunto
    printf "rho(z2,y)^2 + rho(z3,y)^2          = %.6f\n", suma_rho2
end outfile

```

Enlace al guión: [S14-Prct-D-simulados-matricial.inp](#)

Respuestas

1. ¿Por qué la diferencia no es exactamente cero? Por errores de redondeo en aritmética de coma flotante, exactamente la misma razón por la que $\sum \hat{e}_i$ no era exactamente cero en las prácticas anteriores. Invertir una matriz 3×3 y multiplicarla por otro vector implica varias operaciones aritméticas encadenadas, cada una con su pequeñísimo error de precisión; el resultado final coincide con el de `ols` hasta un margen del orden de 10^{-10} o menor, que es indistinguible de cero a efectos prácticos.
2. ¿Por qué necesitamos correlación entre `x2` y `x3`? Porque el objetivo pedagógico de la Actividad 1 es mostrar el caso *general* de la regresión múltiple, en el que resolver el sistema completo (o, equivalentemente, usar la matriz inversa) es imprescindible. Si `x2` y `x3` hubieran resultado casi ortogonales por accidente, el ejercicio se habría parecido demasiado al caso especial de la Actividad 4, y no habría quedado clara la diferencia entre “hay que resolver el sistema” (caso general) y “cada coeficiente se calcula aislado” (familia ortogonal): precisamente el contraste que se explota, con toda intención, entre las Actividades 1-3 y las Actividades 4-5.
3. ¿Por qué la media exactamente nula, no solo la incorrelación? Porque la fórmula de la transparencia “Familia ortogonal” (lección 10) exige que $\mathbf{1}, \mathbf{z}_2, \mathbf{z}_3$ sean *mutuamente* ortogonales: no basta con que $\mathbf{z}_2 \perp \mathbf{z}_3$, hace falta además que cada uno de ellos sea ortogonal a la constante $\mathbf{1}$, es decir, que tenga media cero. Sin esa condición, $\hat{\beta}_1$ (el coeficiente de la constante) no sería, en general, la media de $\mathbf{y}$, y las fórmulas aisladas de $\hat{\beta}_2, \hat{\beta}_3$ como covarianza dividida por varianza dejarían de coincidir con los coeficientes del modelo conjunto.
4. ¿Coincidiría el cálculo aislado con los datos de la Actividad 1? No, en general no coincidiría (salvo coincidencia numérica improbable). La fórmula aislada $\hat{\beta}_2 = \text{cov}(x_2, y) / \text{var}(x_2)$ solo reproduce el coeficiente del modelo conjunto cuando los regresores son mutuamente ortogonales; con `x2` y `x3` correlacionados (como en la Actividad 1), el cálculo aislado ignora por completo la información compartida entre ambos regresores, y por tanto “atribuye” a `x2` parte del efecto que en realidad corresponde a `x3`. Es exactamente el mismo fenómeno que se observó en la práctica de Ramanathan con `sqft` y `bedrms`.
5. ¿Contradice esta práctica la advertencia de la lección 10 sobre la inversa? No, y la distinción es importante. La advertencia de la lección 10 es que el curso nunca usa la inversa como *vía de razonamiento* para justificar por qué MCO funciona, ni como herramienta habitual de cálculo: el argumento conceptual sigue siendo, siempre, la condición de ortogonalidad $\mathbf{X}^\top \hat{\mathbf{e}} = \mathbf{0}$. Aquí hemos usado la inversa una única vez, con un propósito muy distinto: *verificar numéricamente* que una fórmula ya demostrada por otro camino (las ecuaciones normales, obtenidas de la ortogonalidad) da, en efecto, el mismo resultado que `ols`. Usarla como comprobación puntual de una fórmula ya justificada no es lo mismo que apoyarse en ella para *explicar* por qué MCO funciona.